

Vertex Storage Engine Specification

Version: 1.0

Status: Draft

1. Objetivo

O Storage Engine é o subsistema do Vertex responsável por fornecer armazenamento persistente, estruturado e seguro para o Core e plugins.

Seu objetivo é garantir que dados possam ser salvos, consultados, versionados e protegidos de forma consistente em um ambiente altamente modular baseado em plugins.

Ele funciona como camada central de persistência do Vertex.

2. Responsabilidades

O Storage Engine é responsável por:

- armazenar dados persistentes do sistema;
- fornecer armazenamento isolado por plugin;
- gerenciar leitura e escrita de dados;
- versionar esquemas de dados;
- garantir integridade e consistência;
- aplicar políticas de acesso;
- otimizar performance de leitura/escrita;
- integrar com Security Framework;
- gerenciar cache de armazenamento;
- suportar migração de dados.

3. Arquitetura

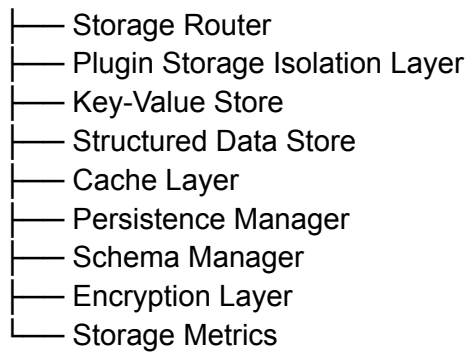
Plugin / Core

↓

Storage API

↓

Storage Engine



↓

Physical Storage (SQLite / Files / Memory-mapped storage)

4. Componentes

Storage Router

Direciona requisições para o tipo de armazenamento adequado.

Plugin Storage Isolation Layer

Garante isolamento de dados entre plugins.

Key-Value Store

Armazena dados simples e rápidos.

Structured Data Store

Armazena dados complexos estruturados (JSON-like, schemas).

Cache Layer

Acelera acesso a dados frequentemente usados.

Persistence Manager

Controla gravação em armazenamento físico.

Schema Manager

Gerencia versões e migração de estruturas de dados.

Encryption Layer

Aplica criptografia em dados sensíveis.

Storage Metrics

Monitora uso e performance.

5. Fluxos

Escrita de dados

Plugin

↓

Storage API

↓

Security Check

↓

Storage Router

↓

Cache / Persistence

↓

Write Confirmed

Leitura de dados

Plugin

↓

Storage API

↓

Cache Lookup

↓

Storage Engine

↓

Return Data

Migração de esquema

Schema Update

↓

Schema Manager

↓

Data Migration

↓

Persistence Layer

↓

Updated Storage

Acesso negado

Plugin Request

↓

Security Framework

↓

Denied Access

↓

Operation Blocked

6. APIs

Armazenamento básico

- put()
- get()
- delete()

Estruturado

- query()
- update()
- list()

Controle

- clear()
- exists()
- size()

Schema

- defineSchema()
- migrateSchema()

Cache

- cacheGet()
- cacheInvalidate()

7. Estados

Estado de almacenamiento

Available

↓

Reading

↓

Writing

↓

Cached

↓

Syncing

↓

Corrupted

↓

Recovered

Estado de datos

- Active
- Archived
- Deleted
- Pending Migration

8. Políticas

Isolamento por plugin

Cada plugin possui namespace próprio de armazenamento.

Segurança obrigatória

Todo acesso passa pelo Security Framework.

Integridade

Dados devem ser validados antes e depois da escrita.

Consistência

Operações devem ser atômicas sempre que possível.

Cache controlado

Cache deve respeitar limites do Resource Manager.

Persistência adaptativa

Dados podem ser promovidos ou removidos do cache automaticamente.

9. Integrações

O Storage Engine integra-se com:

- Security Framework;
- Plugin Manager;
- Plugin Context;

- Execution Runtime;
- Scheduler;
- Resource Manager;
- Memory Management;
- State Engine;
- Event System;
- Logging Framework;
- Configuration System;
- Boot Manager.

10. Métricas

O sistema coleta:

- operações de leitura/escrita;
- latência de acesso;
- uso de cache;
- volume de dados por plugin;
- falhas de escrita;
- corrupção detectada;
- tempo de migração;
- uso de storage físico;
- padrões de acesso.

11. Exemplos

Salvar configuração de plugin

1. Plugin chama "put()".
2. Security Framework valida acesso.
3. Storage Router direciona para namespace.
4. Dados são persistidos.
5. Confirmação retornada.

Leitura rápida via cache

1. Plugin solicita dado.
2. Cache Layer encontra valor.
3. Retorno imediato.
4. Sem acesso ao storage físico.

Migração de dados

1. Schema atualizado.
2. Schema Manager inicia migração.
3. Dados antigos são convertidos.
4. Novo formato armazenado.

Acesso negado

1. Plugin tenta acessar outro namespace.
2. Security Framework bloqueia.
3. Operação é registrada no log.

12. Regras Arquiteturais

- Cada plugin possui isolamento total de dados.
- Acesso entre plugins é proibido por padrão.
- Todas as operações passam por validação de segurança.
- Cache nunca pode violar limites de memória.
- Escrita deve ser atômica quando possível.
- Dados corrompidos devem ser detectados e isolados.

13. Limitações

O Storage Engine não é responsável por:

- execução de código;
- gerenciamento de UI;
- agendamento de tarefas;
- comunicação entre plugins;
- lógica de negócios.

14. Futuras Extensões

O Storage Engine poderá evoluir para:

- sincronização entre dispositivos Vertex;
- armazenamento distribuído;
- compressão automática de dados;

- armazenamento híbrido RAM + disco;
- versionamento histórico completo;
- storage inteligente baseado em uso.

15. Resumo

O Storage Engine é a camada central de persistência do Vertex. Ele fornece armazenamento seguro, isolado e estruturado para plugins e Core, garantindo integridade, performance e escalabilidade. Em conjunto com Security Framework e Resource Manager, forma a base de dados confiável do sistema.